

Tutorial 6: Reinforcement Learning

Q-Learning and Vector-Quantization for Q-Learning (VQQL)

December 9, 2024

- The tutorial aims to learn the basic concepts of Reinforcement Learning, another of the big fields of Machine Learning.
- The tutorial will focus on discrete methods for Reinforcement Learning.
- The tutorial can be done in Linux, Windows or Mac.
- It is important to solve the exercises in order.

1 Introduction

Reinforcement Learning (RL) focuses on training agents to make decisions by interacting with an environment, aiming to maximize cumulative rewards. This tutorial introduces RL through Q-learning, a fundamental tabular method where an agent learns a state-action value table, called a Q-table. While effective for discrete environments, Q-learning struggles with continuous state and action spaces due to the infinite number of possible states and actions.

To address this, we will explore the Vector-Quantization for Q-Learning (VQQL) algorithm, which discretizes continuous spaces to enable Q-table updates. By solving the Frozen Lake and Pendulum problems, you will grasp the transition from classical Q-learning to handling continuous environments with VQQL.

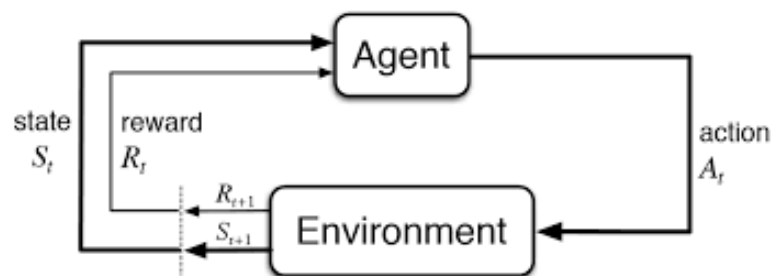


Figure 1: Reinforcement Learning cycle

2 Instructions

You should answer each question in the given order, indicating the exercise number. **The document should not contain screenshots with code.** The data for this tutorial will be the same as in the previous tutorial. Additionally, you will use the *map_1.txt* file from the Frozen Lake tutorials.

3 Exercises

Exercise 1: Classical Q-Learning with Frozen Lake

The Frozen Lake is a discrete problem and thus can be directly solved by classical methods like Q-learning. Q-learning approximates the optimal policy by iteratively updating a Q-table. In previous tutorials, the learning process depended on previously collected data but now we will use Reinforcement Learning, creating an active interaction with the environment. The pseudocode for the Q-learning algorithm is the following:

```
Q-learning (off-policy TD control) for estimating  $\pi \approx \pi_*$   
  
Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$   
Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$   
Loop for each episode:  
  Initialize  $S$   
  Loop for each step of episode:  
    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)  
    Take action  $A$ , observe  $R, S'$   
     $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$   
     $S \leftarrow S'$   
  until  $S$  is terminal
```

Figure 2: Q-learning pseudocode

In this algorithm, you have three different parameters. All of them vary between 0 and 1. It is important to tweak these values to obtain good results.

- Learning rate (α): How much newly acquired information overrides the old information.
- Discount factor (γ): The importance of future rewards compared to immediate rewards.
- Exploration rate (ϵ): The probability of choosing a random action over the best-known action.

1. Implement the Q-learning algorithm in Python and all the necessary functions to evaluate the agent. Make sure that the update process is correct.
2. Train the agent to solve the Frozen Lake map using different values for the parameters mentioned. Evaluate the agent's performance periodically and obtain the accumulated reward. Compare the results in a table.
3. Select the best parameter configuration and deploy the agent. Analyze the learned policy. Does the agent successfully avoid holes and reach the goal?

Exercise 2: Vector-Quantization for Q-Learning (VQQL) with Pendulum

The Pendulum problem involves continuous state and action spaces, where Q-learning's reliance on a finite Q-table becomes impractical. VQQL extends Q-learning by discretizing these spaces, enabling the algorithm to work effectively in such environments. This exercise introduces VQQL and applies it to stabilize the pendulum upright. The parameters to tune:

- State space quantization: Choose the number of bins or clusters for discretizing the continuous state space
- Action space quantization: Choose the number of bins or clusters for discretizing the continuous action
- Learning rate (α): We will keep this value to 0.1
- Discount factor (γ): We will keep this value to 0.99
- Exploration rate (ϵ): We will keep this value to an epsilon decay from 1 to 0.

1. Modify the code for the Q-learning agent such that it discretizes the state and action. Modify the necessary functions.

2. Use the files *uniform_states.csv* and *uniform_actions.csv* from Aula Global. These files contain some predefined clusters for the Pendulum problem using a uniform discretization. Train the agent to solve the Pendulum problem using all these discretizations. Evaluate the agent's performance periodically and obtain the accumulated reward.
3. Now obtain the discretized states and actions for different numbers of clusters (between 4 and 2048 for the state space and 2 and 64 for the action space) using the K-means algorithm from the previous tutorial. Save all the clusters into CSV files.
4. Repeat the training process for all the new combinations and evaluate the agent's performance to obtain the cumulative reward.
5. Select the best discretization combination for the VQQL. Analyze and compare its performance against the uniform discretization. Do the agents successfully solve the problem? Does the K-means discretization provide any benefits?

4 Files to Submit

All the lab assignments **must** be done in groups of 2 people. You must submit a .zip file containing the required material through Aula Global before the following deadline: **Sunday, 22th December 2024 at 12:00**. The name of the zip file must contain the last 6 digits of both student's NIA, i.e., `tutorial6-123456-234567.zip`

The zip file must contain the following files:

1. A **PDF** document with (8 page limit without cover page):
 - Cover page with the names and NIAs of both students.
 - Answers to all the questions appearing in the Exercises section.
 - Description of the parameters used in each case are the selected values.
 - Explanation of the decisions made to select the best models.
 - Conclusions.
2. Include in a folder called "material" the following files:
 - The files used to implement all functionalities of the tutorial.

Please, **be very careful and respect the submission rules**.